

datalab 报告

姓名：王奕萱

学号：2025201745

总分	bitAnd	bitXor	samesign	logtwo	byteSwap	reverse
37/37	1/1	1/1	2/2	4/4	4/4	3/3

logicalShift	leftBitCount	float_i2f	floatScale2	float64_f2i	floatPower2
3/3	4/4	4/4	4/4	3/3	4/4

test 截图：

```
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:    PASS    detail: Pass using 4 operations including {'|', '~'} and {'int'}.
bitXor      1/1:    PASS    detail: Pass using 7 operations including {'&', '~'} and {'int'}.
samesign    2/2:    PASS    detail: Pass using 9 operations including {'&&', '>>', '!', '^'} and {'int'}.
logtwo      4/4:    PASS    detail: Pass using 18 operations including {'|', '<<', '>', '>>'} and {'int'}.
byteSwap    4/4:    PASS    detail: Pass using 10 operations including {'&', '<<', '>>', '^'} and {'int'}.
reverse     3/3:    PASS    detail: Pass using 23 operations including {'&', '|', '<<', '>>'} and {'unsigned'}.
logicalShift 3/3:    PASS    detail: Pass using 6 operations including {'&', '<<', '>>', '~'} and {'int'}.
leftBitCount 4/4:    PASS    detail: Pass using 32 operations including {'~', '!', '+', '>>', '<<'} and {'int'}.
float_i2f    4/4:    PASS    detail: Pass using 19 operations including {'>', '~', '!', '+', '==', '<', '&', '>>', '<
<', '-'} and {'int', 'while', 'unsigned', 'if'}.
floatScale2  4/4:    PASS    detail: Pass using 11 operations including {'|', '+', '==', '&', '>>', '<<'} and {'unsig
ned', 'if'}.
float64_f2i  3/3:    PASS    detail: Pass using 18 operations including {'|', '>', '~', '+', '<', '&', '>>', '<<', '-
'} and {'int', 'if', 'unsigned'}.
floatPower2  4/4:    PASS    detail: Pass using 9 operations including {'<', '+', '<<', '-'} and {'int', 'unsigned',
'if'}.
Total points: 37
(base) yxxxx@xxxx:~/CSAPP/datalab2026$
```

解题报告

亮点

- 1. logtwo
- 2. logicalShift
- 3. leftBitCount
- 4. float_i2f

logtwo 排在最前，是因为它可用的运算符只有 `> < >> << |`，累加的写法只能另找。logicalShift 用六个运算符完成，掩码的构造绕开了 `n` 取 0 时的移位越界。leftBitCount 与 logtwo 共用同一套二分骨架。float_i2f 要处理的边界情形最多。

bitAnd

可用的运算符是取反与按位或。德摩根律把与转成或， $x \& y$ 等于 $\sim(\sim x \mid \sim y)$ ，四个运算符。

bitXor

$x \wedge y$ 取 1，当且仅当 $x \mid y$ 取 1 而 $x \& y$ 取 0，于是异或可以写成 $(x \mid y) \& \sim(x \& y)$ 。这一题只给了取反与按位与，其中的 $x \mid y$ 要再用一次德摩根律换成 $\sim(\sim x \& \sim y)$ ：

```
return  $\sim(\sim x \& \sim y) \& \sim(x \& y)$ ;
```

四个取反加三个按位与，正好是上限的七个。

samesign

题目把 0 单独算一类，符号位只能给出一半的答案。0 与正数的符号位同为 0，题目却把它们分在两类。

判据因此拆成两项相与。 $(x \gg 31) \wedge (y \gg 31)$ 取 0 说明符号位相同， $(!x) \wedge (!y)$ 取 0 说明两数“是否为零”这一属性相同。负数的符号位恒为 1，所以第二项只在两数符号位同为 0 时起区分作用，正好把 $\text{samesign}(0, 1)$ 与 $\text{samesign}(0, 0)$ 分开。

logtwo

$\text{floor}(\log_2(v))$ 就是 v 最高位 1 的下标，求它的常规办法是二分。先看 v 是否超过 $0xFFFF$ ，超过则结果的第 4 位为 1，同时把 v 右移 16 位；接着依次比较 $0xFF$ 、 $0xF$ 、3，对应结果的第 3、2、1 位。四轮之后 v 落在 1 到 3 之间， $v \gg 1$ 给出结果的第 0 位。

这一题可用的运算符里缺少加法，累加要换个写法。 $s = (v > 0xFFFF) \ll 4$ 让比较结果直接变成这一轮该记的权重，而 16、8、4、2、1 分别占据结果的一个二进制位，彼此独立，因此按位或与加法在这里结果相同。

byteSwap

两个字节要互换，取出再拼回需要多个掩码。这里改用异或。设 d 为两个字节的差异，

```
int d =  $((x \gg nb) \wedge (x \gg mb)) \& 0xFF$ ;
```

把 d 同时异或回两个位置，两个字节即互换。 $\& 0xFF$ 把差异限制在一个字节内，同时消掉 x 为负数时算术右移带来的符号扩展。 n 与 m 相等时 d 为 0，返回值即 x 本身。

reverse

32 位整体倒序，逐位搬运要循环 32 次。蝶形交换把它压到五轮：先交换相邻的单个位，再依次交换 2 位块、4 位块、8 位块，最后交换两个 16 位半字。每一轮的形状相同，

```
v = ((v >> k) & M) | ((v & M) << k);
```

掩码依次取 `0x55555555`、`0x33333333`、`0x0F0F0F0F`、`0x00FF00FF`。最后一轮写成 `(v >> 16) | (v << 16)`，移位本身已经把另一半挤了出去，掩码可以省去。`v` 的类型是 `unsigned`，右移为逻辑移位，高位补 0。

logicalShift

`x` 是 `int`，`x >> n` 为算术右移，高 `n` 位被符号位填满，要用掩码清掉：

```
return (x >> n) & ~(((1 << 31) >> n) << 1);
```

`(1 << 31) >> n` 得到 `n + 1` 个高位 1，左移一位变成 `n` 个，取反即得低 `32 - n` 位为 1 的掩码。多绕的这一步是为了 `n` 取 0：若直接写 `(1 << 31) >> (n - 1)`，`n` 为 0 时移位量为 -1，属于未定义行为。

leftBitCount

要数的是从最高位起连续 1 的个数。`~x >> k` 取 0，当且仅当 `x` 的高 `32 - k` 位全为 1：取反后这些位变成 0，算术右移 `k` 位的结果便是 0。`!(~x >> k)` 于是成为“高位是否全 1”的判据。

拿它做二分。先试高 16 位，成立就记 16 并把 `x` 左移 16 位，剩下的部分继续；接着是 8、4、2、1，最后补一次单位判断。每一步的形状相同，

```
c = (!(~x >> k)) << j;  
n = n + c;  
x = x << c;
```

计数上界为 `16 + 8 + 4 + 2 + 1 + 1 = 32`，对应 `leftBitCount(-1)` 的返回值。这套骨架与 `logtwo` 一致，判据从“是否大于某个数”换成了“高位是否全 1”。

float_i2f

`int` 转 `float` 要定出符号、阶码与 23 位尾数。符号取自 `x` 的最高位；阶码和尾数来自 `|x|` 的二进制形式，把最高位的 1 对齐到 bit31 之后，阶码随之确定。

```

m = |x| // 放在 unsigned 里
e = 158 // 127 + 31, 对应最高位落在 bit31
while 最高位为 0: m <= 1, e -= 1
tail = m & 0xFF // 23 位尾数之外被移出去的部分
m = (m >> 8) & 0x7FFFFFFF // 舍去隐含的那个 1
按 tail 舍入
return s + (e << 23) + m

```

x 取 `INT_MIN` 时求绝对值会溢出。`-x` 与 `int` 域内的 `~x + 1` 对 `0x80000000` 都是有符号溢出，属于未定义行为。所以先把 x 赋给 `unsigned` 的 `m`，再在无符号域里算 `~m + 1`，回绕是有定义的，结果为 `0x80000000`，正是 2^{31} 这个幅值。

`float` 只有 23 位尾数，移出去的低 8 位按就近舍入、中间值取偶处理。`tail` 超过 128 进位，等于 128 时看当前尾数的奇偶，即 `m + (m & 1)`，小于 128 则舍去。最后一行用加法拼接，原因是尾数进位溢出成 `0x800000` 时，加法会自然进到阶码字段上。 x 取 `0x01FFFFFF` 时有效位是 25 位，舍入后尾数由 `0x7FFFFFFF` 变为 `0x800000`，阶码由 151 变为 152，结果是 2^{25} ，符合中间值取偶的要求。

floatScale2

浮点数乘 2 即阶码加一，边界处另有分支。

阶码	处理
<code>0xFF</code>	Inf 或 NaN，按题目原样返回
0	非规格化数，尾数左移一位，符号位单独或回来
<code>0xFE</code>	翻倍后溢出成 Inf，返回 符号 \ <code>0x7F800000</code>
其余	阶码加一，即 <code>uf + 0x800000</code>

非规格化那一支的进位是自动的。尾数最高位左移一位正好落进阶码的最低位，得到的是阶码为 1 的规格化数，这正是它该有的值。

`0xFE` 要单列。若它也走 `uf + 0x800000`，阶码会变成 `0xFF` 而尾数原封不动，尾数只要带 1，结果就成了 NaN。

float64_f2i

`double` 的 53 位尾数跨了两个 32 位字，高字 `uf2` 带 20 位，低字 `uf1` 带 32 位。记 $E = \text{exp} - 1023$ ，整数部分即 53 位尾数右移 $52 - E$ 位。

E 小于 0 时绝对值小于 1，向零截断的结果是 0。 E 大于 30 时溢出，返回 `0x80000000`；Inf 与 NaN 的阶码为 `0x7FF`， E 为 1024，也落在这一支。 E 等于 31 时只有 -2^{31} 可以表示，而它的答案同样是 `0x80000000`，所以这一支可以并进去。

其余情形要看 E 是否超过 20。 E 在 20 以内时整数部分全在高字里， $m \gg (20 - E)$ 即可； E 超过 20 时把高字左移 $E - 20$ 位，再拼上低字的高 $E - 20$ 位。取的是幅值，右移天然向零截断，最后按 $uf2$ 的符号位决定是否 $\sim r + 1$ 。

floatPower2

2.0^x 落在哪一段由 x 决定。 x 在 128 以上时阶码超过 254，返回 $+INF$ 。 x 在 -126 到 127 之间是规格化数，阶码为 $x + 127$ ，尾数为 0。 x 在 -149 到 -127 之间掉进非规格化区，此时值为 $2^{-126} \times \text{frac} / 2^{23}$ ，令它等于 2^x 解出 $\text{frac} = 1 \ll (x + 149)$ 。 x 在 -149 以下时已经小于最小的非规格化数，返回 0。

三个 `if` 按从小到大排列，每一支只比较上界，下界的比较由前一支承担。

反馈/收获/感悟/总结

整体花了一个下午。前面几题是 CSAPP 第二章的直接应用，占时间的是 `logtwo` 与 `float_i2f`。前者被可用的运算符集合逼着重新想累加的写法，后者先写出能过 `./btest` 的版本，再逐个补边界。

`./btest` 只检验正确性，运算符的数量要另外数。`bitXor` 第一版正确性满分而运算符用了八个，超出上限，重写之后压到七个。后面的题都是先数一遍运算符再跑脚本。

参考的重要资料

- 《深入理解计算机系统》第 2 章，2.2 整数表示与 2.4 浮点数。阶码偏移、非规格化数的取值范围，以及 IEEE 754 的舍入规则取自这里。
- Sean Eron Anderson, *Bit Twiddling Hacks*, <https://graphics.stanford.edu/~seander/bithacks.html>。`logtwo` 的二分求最高位来自其中的“Find the log base 2”，`reverse` 的蝶形掩码序列来自“Reverse bits in parallel”。
- IEEE 754-2008 中 `round-to-nearest-even` 的定义，用于 `float_i2f` 的中间值取偶。